Notes for Distributed Systems

MIRI-CPDS

Yuechao Liu
Universitat Politècnica de Catalunya

2026-10-20

Contents

1 Failure Models 4
2 Global Time 4
2.1 Physical Clocks 4
2.1.1 External Synchronisation: Christian's Algorithm 5
2.1.2 Internal Synchronisation: Berkeley Algorithm 5
2.1.3 Time Stepping (TODO!) 6
2.2 Logical Clocks 6
2.2.1 Happen-before Relation 6
2.2.2 Lamport's Logical Clock 6
2.2.3 Vector Clock 7
2.2.4 Dotted Version Vector ???!!!! 8
3 Global States 8
3.1 Basic Concepts 8
3.2 Linearisation 9
3.3 Chandy-Lamport Shapshot Algorithm (Global State) 9
3.4 Global State Predicates 11
3.4.1 State Predicate 11
3.4.2 Not-stable 11
4 Election Algorithms 12
4.1 Bully Algorithm 12
4.2 Chang and Robert's Ring Algorithm 13
4.3 Enhanced Ring Algorithm 14
5 Multicast Communication 14
5.1 Reliable Multicast 15
5.1.1 Basic Reliable Multicast 15
5.1.2 Scalable Reliable Multicast 15
5.2 Ordered Multicast 15
5.2.1 FIFO Ordering (within each process) 15
5.2.2 Causal Ordering (between processes) 16
5.2.3 Total Ordering (all processes) 16
5.2.3.1 Sequencer Implementation 16
5.2.3.2 Joint Agreement Implementation 17
5.3 Atomic Multicast (View-synchronous Multicast / Virtual Synchrony) 17
5.4 Consensus 18
5.4.1 Unreliable Communication (Two-army Problem) 19
5.4.2 Crash-faulty Processes, Synchronous System 19
5.4.3 Byzantine-faulty (Lying) Processes, Synchronous System 19
5.4.3.1 Oral Message Algorithm (OM) !!!! 20
5.4.4 Faulty Process, Asynchronous System 21
5.4.4.1 Fault Masking + Basic Algorithm 21
5.4.4.2 Failure Detectors + Synchronous Algorithm 22
5.4.4.3 PAXOS (Quorum) 22
6 Concurrency 22
6.1 Serially Equivalent 23
6.2 Strict Two-phase Locking 23
6.2.1 Deadlock Detection 23
6.3 Timestamp Ordering 24

6.3.1 Write Validity 24
6.3.2 Read Validity 25
6.4 Optimistic Concurrency Control 25
6.5 Backward Validation 26
6.6 Forward Validation 26
7 Distributed Transactions 26
7.1 Distributed Commit 26
7.1.1 Two-Phase Commit Protocol 26
7.1.2 Other Protocols 27
7.2 Distributed Concurrency Control 27
7.2.1 Strict Two-Phase Locking (Distributed) 27
7.2.1.1 Distributed Deadlock Detection 28
7.2.1.1.1 Centralised Detection 28
7.2.1.1.2 Edge Chasing 28
7.2.2 Timestamp Ordering (Distributed) 29
7.2.3 Optimistic Concurrency Control (Distributed) 29
8 Consistency 29
8.1 Consistency Model: Distributed Data Store 29
8.2 Data-centric Consistency 30
8.2.1 Strict Consistency 30
8.2.2 Linearisability (Strong Consistency) 30
8.2.3 Sequential Consistency 31
8.3 Client-centric Consistency 31
8.3.1 Read Your Writes (RYW) 32
8.3.2 Monotonic Reads (MR) 33
8.3.3 Monotonic Writes (MR) 33
8.3.4 Writes Follow Reads (WFR) 33
8.4 Consistency Protocol 33
8.4.1 Primary-based Protocols 33
8.4.1.1 Primary-backup Remote-write 33
8.4.1.2 Primary-backup Local-write 34
8.4.2 Replicated-write Protocols 34
8.4.2.1 Active Replication Protocol 34
8.4.2.2 Quorum-Based Protocols 35
8.4.2.2.1 Quorum Write 35
8.4.2.2.2 Quorum Read 35

1 Failure Models

- Crash Failure: Process halts and remains halted
  - Fail-stop crash: other processes can reliably detect its failure
  - Fail-silent crash: other processes cannot reliably detect its failure
- Omission Failure: Sent message **NEVER** arrive at the other end
  - Receive Omission
  - Send Omission
- Timing Failure: Process response lies outside a specific time interval
- Response Failure: Process response it incorrect
  - Value Failure: the value of the response is wrong
  - State transition failure: The process deviates from the correct control flow
- Arbitrary (Byzantine) Failure: Process may arbitrarily omit intended steps or take unintended steps (exhibit differently to different observers)

2 Global Time

2.1 Physical Clocks

1. Has an accuracy of milliseconds (ms)
2. Perfect synchronisation is not feasible
3. Cannot be used to order events

Physical Clock Synchronisation

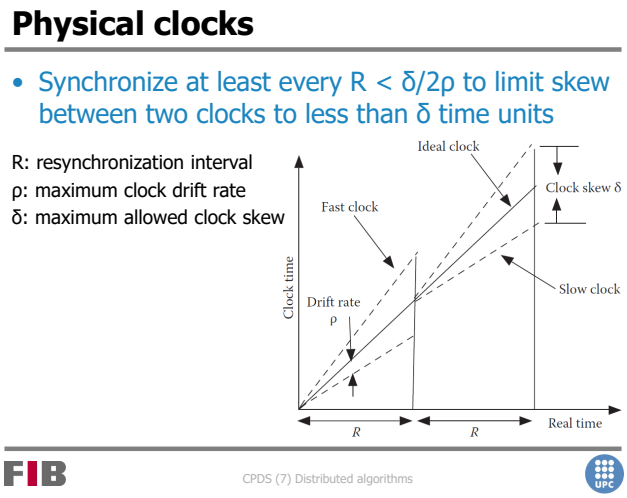


Figure 1: Resynchronisation Interval: Problem Statement

Given:

1. maximum clock drift ρ, and
2. maximum clock skew δ

Want to find: Resynchronisation interval R

Note that the rate for the "perfect" clock is 1, i.e. the "perfect" clock does not drift.

Since the clock can drift faster or slower, we need to make sure the time difference between the **slowest** clock $R \cdot (1 - \rho)$ and the **fastest** clock $R \cdot (1 + \rho)$ does not diverge more then δ, hence we have the following relationship.

$$\begin{aligned} \Delta t = t_{\text{fast}} - t_{\text{slow}} = R \cdot ((1+\rho) - (1-\rho)) = 2R\rho \leq \delta \\ \Rightarrow R \leq \frac{\delta}{2\rho} \end{aligned} \tag{1}$$

2.1.1 External Synchronisation: Christian's Algorithm

Christian's Algorithm synchronises **nodes** (C, client) with a **UTC server** (S, Server) within a certain bound. **Assume**:

1. Latency is symmetrical: $T_{C\to S} = T_{S\to C}$

Given:

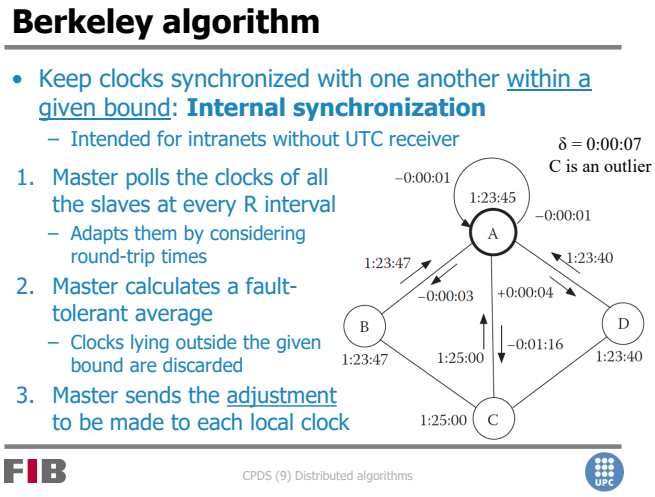
1. Round-trip time (RTT): The time duration between the sending and receiving of the message from/to the client. In this figure, it's $T_2 - T_1$.

Note:

1. Here, we only interested in the time between when the server responds and the client receives the message, i.e. $T_{S\to C} = \frac{\text{RTT}}{2}$. (Since we assume that the latency is symmetric)
2. We cannot know when the server will receive the request and when it will respond. So the server response time $T_s' \leq T_s \leq T_s''$
3. The accuracy of the clinet is $\pm\frac{\text{RTT}}{2} - \text{Min}$, which is $T_s - T_s'$ or $T_s - T_s''$. (Since $T_2 - T_s = \frac{\text{RTT}}{2}$, $T_s'' - T_2 = \text{Min}$)

2.1.2 Internal Synchronisation: Berkeley Algorithm

Berkeley's Algorithm synchronises nodes with one another within a certain bound.



Assume:

- We are given a tolerance of δ

Algorithm:

1. Master pulls each client at each R interval
   - Master adapts them by considering RTT.
   - The slide did not count for RTT?
2. Master calculate a fault-tolerance average
   - Slaves lying outside the given bound is discarded
3. Master sends the adjustment to be made to each local clock

2.1.3 Time Stepping (TODO!)

TODO

2.2 Logical Clocks

2.2.1 Happen-before Relation

We use the happen-before relation to capture **EVENTS** E.

- Events: happen in processes, e.g. sending and receiving of messages (The dots in the lines)
- Messages: Sent across processes (the arrows between dots)

Assume

- A set of processes P
- A set of **ALL** Events E
  - Events in process $P_i \in P$: $E(P_i) \subset E$
- Timestamp of Event $e \in E$: $C(e)$
- Timestamp of message m: $\text{ts}(m)$

1. **Same process**: If $a, b \in E(P_i)$ and a comes before b, then $a \to b$
2. **Across processes**: If $a \in E(P_i)$ is the sending event, and $b \in E\big(P_j\big)$ $(i \neq j)$ is the receiving events, then $a \to b$
3. **Transitivity**: $a \to b \wedge b \to c \Rightarrow a \to c$

Use timestamp to capture the happen-before relationship $a \to b \Rightarrow C(a) < C(b)$ where $C(e)$ is the timestamp to event e.

2.2.2 Lamport's Logical Clock

Lamport's Clock defines a **Partial Order** that is consistant with the *happen-before* relation.

If we want total order, use Lamport's clock but add the process ID $(C_i(a), i)$, so that $(C_i(a), i) < \big(C_j(b), j\big) \Leftrightarrow \big(C_i(a) > C_j(b)\big) \vee \big(C_i(a) = C_j(b) \wedge i > j\big)$.

Given

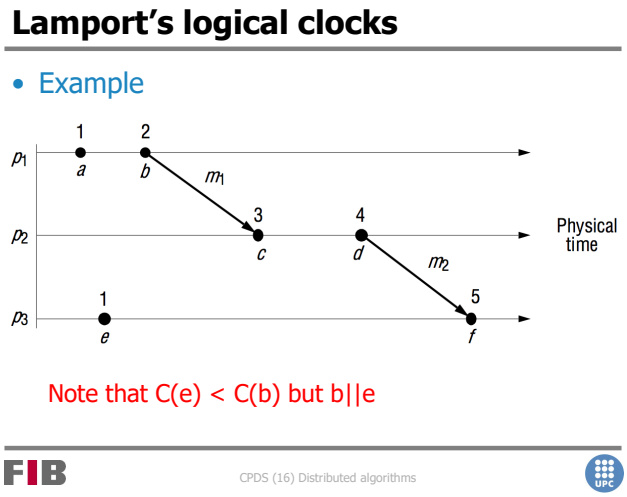
1. Each process $P_i \in P$ has a counter C_i
2. This C_i is used as the **timestamp** attached to **EVENTS** in P_i, namely $e \in E(P_i) \subset E$
3. $C(m)$: timestamp for the message

It does not guarantee causation:

- $a \to b \Rightarrow C(a) < C(b)$
- Not otherwise (b and a could be parallel)
  - $C(a) < C(b) \Rightarrow (b \to a) \vee (b \parallel a)$

Algorithm

1. When an **EVENT** happens in P_i, increase the counter C_i
   - What do we mean by an **EVENT** happens in P_i
2. When P_i sends a message m to P_j, attach C_i as the timestamp
   - $C(m) := \text{ts}(m) \leftarrow C_i$
3. When P_i received a message m from P_j
   - $C_i \leftarrow \max(C(m) := \text{ts}(m), C_i) + 1$



2.2.3 Vector Clock

Vector Clock enhances Lamport's Clock to allow causality. The idea is to use a per-process vector to track at current process P_i, the version numbers of all processes P_j.

Given

- a set of processes P
- a set of events E

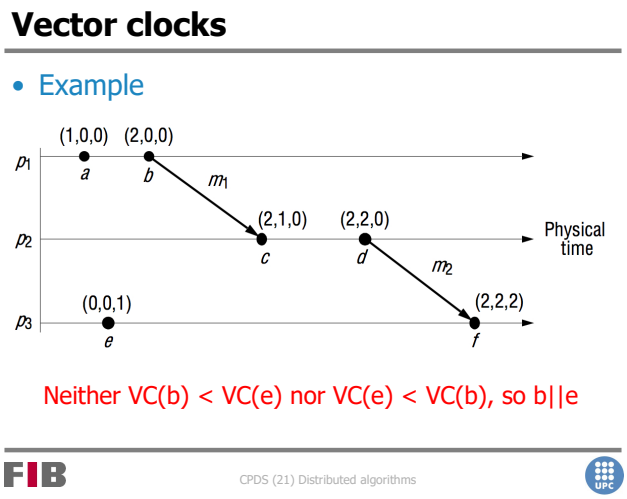
Construction

Each process $P_i \in P$ maintains a **Vector Clock** VC_i where

- An array of counters for each processes: $|\mathrm{VC}_i| = |P|$
- $\mathrm{VC}_i[j]$: Process P_i tracks its progress from events coming from P_j
- $\mathrm{VC}(m)$: The vectored timestamp for message m
- Comparison $\mathrm{VC}(a) < \mathrm{VC}(b)$:
  - iff ALL of their elements are "<=" and EXISTS some elements that is strictly "<"
  - $\mathrm{VC}(a) < \mathrm{VC}(b) \Leftrightarrow (\forall k : \mathrm{VC}(a)[k] \leq \mathrm{VC}(b)[k]) \wedge (\exists k : \mathrm{VC}(a)[k] < \mathrm{VC}(b)[k])$
- Causality
  - $\mathrm{VC}(a) < \mathrm{VC}(b) \Leftrightarrow a \rightarrow b$
- Parallel
  - $\neg(\mathrm{VC}(a) < \mathrm{VC}(b) \vee \mathrm{VC}(a) > \mathrm{VC}(b)) \Rightarrow a \parallel b$
  - We can simplify the above expression to:
    - $\exists k_1, k_2 : (\mathrm{VC}(a)[k_1] < \mathrm{VC}(b)[k_1]) \wedge (\mathrm{VC}(a)[k_2] > \mathrm{VC}(b)[k_2]) \Rightarrow a \parallel b$

Algorithm

- When P_i send a message m
  1. $\mathrm{VC}_i[i] \leftarrow \mathrm{VC}_i[i] + 1$
  2. $\mathrm{ts}(m) \leftarrow \mathrm{VC}_i$
- When P_i receives an message m from P_j (an event $e \in E(P_i)$)
  1. $\forall k \in 0..|P| : \mathrm{VC}_i[k] \leftarrow \max(\mathrm{VC}_i[k], \mathrm{VC}(m))$
  2. $\mathrm{VC}_i[i] \leftarrow \mathrm{VC}_i[i] + 1$



2.2.4 Dotted Version Vector ???!!!!

The version vectors are used in distributed systems for conflict resolution on concurrent update. The version vector consists of $((\text{id}, \text{CausalPastMaxID}), \text{PresentEventID})$.

It keeps the "present event" (the dot) separate from the "causal past". Hence, if the "present event" of X **is contained in** the "causal past" of Y, it means that $X \to Y$ (Y is generated causally following X).

In the slides, we are considering the version vectors for a **SINGLE** key, we simplified the notation

$$C[(r, i, j)] = \{r_j\} \cup \{r_1, ..., r_n\} \tag{2}$$

Construction

- We have a set of replicas R (as opposed to processes P)
- Items are the "entires" in each database, for example.
- Each replica R_k (or k) keeps its own version vector for each item $\text{VV}_k = \{((r_l, i_l), j_l) \in \mathbf{VV}\}$ where r_l is the replica, i_l is the range of events $\{1, ..., i_l\}$ and j_l is the last event.
  - For example, $((c, 2), 4) \in \mathbf{VV}$ represents events in replica c: $\{c_1, c_2\} \cup \{c_4\}$, hence
  - $\{(a, 2), (b, 1), ((c, 2), 4)\} \equiv \{a_1, a_2, b_1, c_1, c_2, c_4\} \subset 2^{\boldsymbol{E}}$
- Each incoming write (message) include a vector of **prior read items** (aka, causal context)
  - VV_{inc}
  - Intuition: This write to item I_i is based on reading items $I_{\text{context}} \subset 2^I$

Order

Causality $a \to b$:

TODO

3 Global States

3.1 Basic Concepts

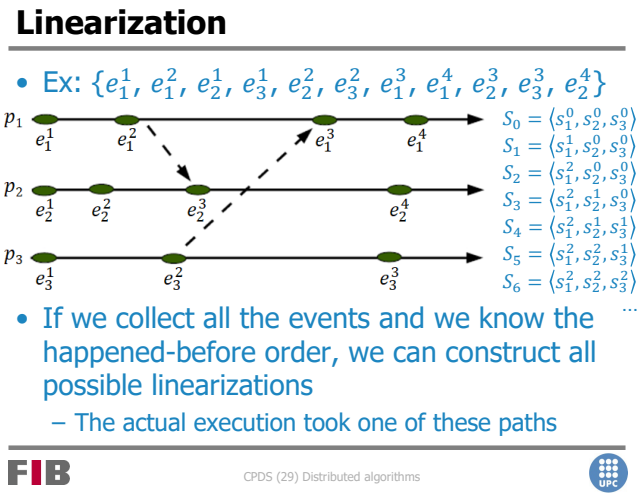
- **History of Process**: The sequence of events the occour at a process $h_i =< e_i^1, e_i^2, ... >$.
- **State of process**: The **finite prefix** of the history after k events, i.e. $s_i^k =< e_i^1, e_i^2, ..., e_i^k >$.
- **Global history**: The union of individual processes $H = \bigcup_i h_i$.

- **Cut**: The global history up to a specific event in each process history.
  - union of prefixes of process histories
  - e.g. $S =< s_1^{k_1}, s_2^{k_2}, s_3^{k_3}, ..., s_N^{k_N} >$ where $s_i^k =< e_i^1, ..., e_i^k >$.
  - Consistent cut: for each event, it contains all events that **happen-before**.
- **Run**: a **total ordering** of all events in a global history consistent with each **local history** (maintain $\forall i : e_i^{k_1} < e_i^{k_2} \Leftrightarrow k_1 < k_2$).

3.2 Linearisation

A run consistant with the **happen-before relation** (message passing, transitivity)

Reachability of States: State S' is reachable from S if and only if there is a linearisation passing through s and then s'.



In the above figure, we have

- a run of $\{e_1^1, e_1^2, ...\}$ that is consistant with the happen-before relationship as listed on the top, and
- a potential state transition $S_0 \to S_1 \to ... \to S_6$ as listed on the left.
  - Note that each state transition only alters one column, e.g. $S_0 \to S_1$ only alters $s_1^0 \to s_1^1$, which executes event e_1^1,

3.3 Chandy-Lamport Shapshot Algorithm (Global State)

An algorithm that allows each processor to take **individual snapshots** s.t. when the snapshot is later stitched together by **some external** to the algorithm, it would be consistent.

It does not care how to put individual snapshots together.

- Processes may continue to their execution while taking the snapshot
- Any process may initiate a snapshot at any time (decentralised algorithm)
- Separate snapshots may be initiated concurrently (tagged with a marker message)

Assume:

- **Reliable** processes and channels
- **Unidirection channels** $\forall i \neq j : C_{i,j} \Leftrightarrow C_{j,i} \in$ Channels
- **FIFO channels**: Arrives in sending order!
  - no message crossing, e.g. P_i send (m_1, m_2) and P_j receives (m_2, m_1).
- **Strongly connected topology**

- does not require fully connected
- the algorithm **flood** through the network (since the receiving sends marker to its neighbours acting as somehow another "initiator" if its neighbour hasn't seen a message).

When the algorithm terminates, **everyone** would have sent **everyone** a marker message.

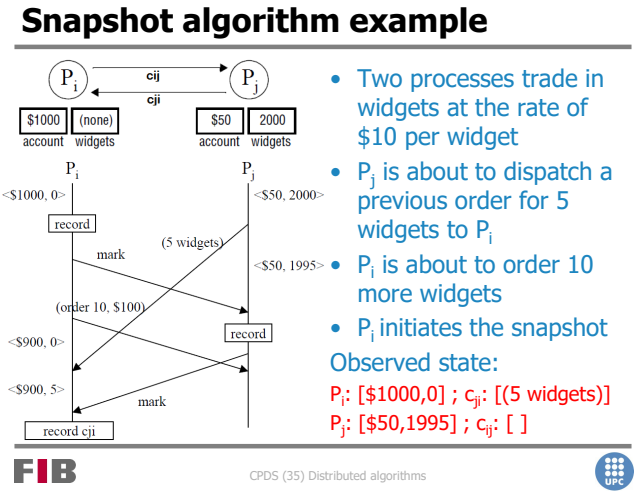
Algorithm for Initiator Kick-start i (On Recording Request)

1. Record its own state $S_i^n(P_i)$ (All events on P_i so far)
2. Send a special marker on **every** outgoing channels: $\forall j$: send$\big(C_{i,j}, \text{Marker}\big)$
3. Start recording messages on **each incoming channels** until each of its incoming channels has received a marker
   - $\forall k$: receive$\big(C_{k,i}, \text{Marker}\big)$

Algorithm for Receiver AND Initiator j (On Receiving a Marker from $C_{i,j}$)

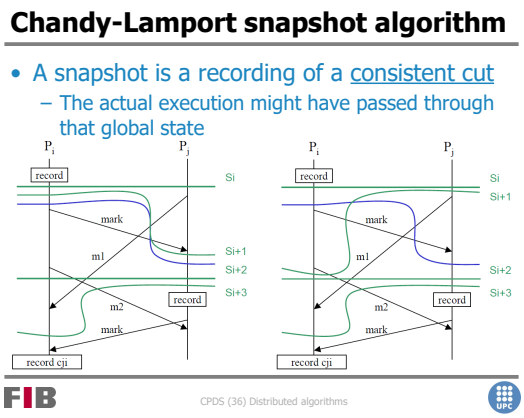
- **P_j has not saved its state**
  1. Record its own state $S_j^m(P_j)$.
  2. **Mark the channel $C_{i,j}$ as empty** (because we saved the states)
     - If we receive the marker message, it means that **ALL** messages before the snapshot has arrived (on this specific channel)
     - If we receive any messages from this "empty" channel, it must be in the "future" (FIFO ordering), we exclude this from our snapshot
     - This marks the **end of snapshot of the current $C_{i,j}$ channel**, now we want to take snapshot of other channels
  3. Send special marker on **every outgoing** channel: $\forall k$: send$\big(C_{j,k}, \text{Marker}\big)$
     - This includes the channel $C_{j,i}$ to reply to process P_i
  4. Start **recording** message over **OTHER incoming channels** until each of the incoming channels have received a marker.
     - $\forall k \neq j$: receive$\big(C_{k,j}, \text{Marker}\big)$
     - The state of channel $C_{k,j}$ is the sequence of all messages from the time P_j records its state until the time marker arrive from P_k
- **P_j has saved its state**
  1. P_j stop recording on $C_{i,j}$ and set $C_{i,j}$s final state as the sequence of all incoming messages that arrive on $C_{i,j}$ since recording begin

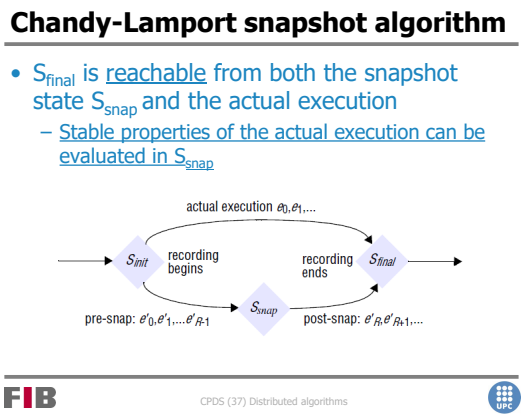
Example



The above figure describes a simple scenario for channels $C_{i,j}$ and $C_{j,i}$.

- **For P_i**
  1. P_i record its state $S_i^n(P_i)$ (in this case it would be $< \$1000, 0 >$)
  2. P_i send marker to P_j through $C_{i,j}$ **It now has seen A MARKER** (because it sends one)
  3. P_i start recording messages: $S_i^n = \varnothing$
     1. P_i receives $\text{msg}_1 = (\text{order } 10, \$100)$, records it, now $S_i^n(C_{j,i}) = (\text{msg}_1)$
     2. P_i receives marker from P_j, since it has seen a marker, stop recording
        - The final state for $C_{j,i}$: $S_i^n(C_{j,i}) = (\text{msg}_1)$.
  4. Terminates since it has seen markers on each of its incoming channels
- **For P_j**
  1. P_j send message $\text{msg}_1 = (\text{order } 10, \$100)$ to P_i through $C_{j,i}$.
  2. P_j receives a marker from P_i, it is the **FIRST TIME** P_j has seen a marker from $C_{i,j}$ hence $S_j^m = \varnothing$
  3. P_j send a marker to each of its channels, in this case: $C_{j,i}$.
  4. Terminates since it has seen markers on each of its incoming channels





3.4 Global State Predicates

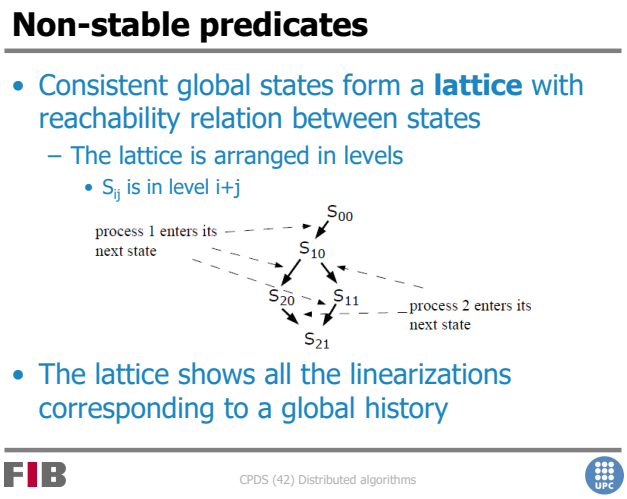
3.4.1 State Predicate

- A predicate become true and ti remains true for **all reachable states**
- If a stable predicate is true in a snapshot then it is also true in the execution

3.4.2 Not-stable

Lattice

- Consistent global states form a **lattice** with reachability relation between states
- The lattice shows all the linearisation corresponding to a global history



- A predicate can evaluate true and then become false in some **reachable state**
- Category
  - Possibly Φ
    - $\exists$ consistent global state S as part of a linearisation such that $\Phi(S) = T$
    - Check through all consistent global states of the lattice to check whether Φ is true for all of them
      - $\exists S \in \text{Lattice} : \Phi(S) = T$
  - Definitely Φ
    - $\forall$ linearisation pass through some consistent global state S such that $\Phi(S) = T$
    - Step through all the paths **from the initial to the final** state of the lattice to check if **all of them** include a consistent global state for which Φ is true
      - $\forall p \in \text{Path} : \exists S \in p : \Phi(p) = T$
  - Never Φ
    - $\neg\exists$ linearisation pass though **ANY** global state S such that $\Phi(S) = T$
    - Step through all the consistent global states of the lattice to check whether Φ is false for all of them

4 Election Algorithms

Leader election algorithms ensure that an election includes with **all processes** agreeing on a **unique coordinator**.

Just pick one, WLOG, choose the one with largest identifier. (identifiers must be unique and totally ordered)

The following algorithms **CANNOT** tolerate network partition

Properties

- **Safety**: a participant is either **non-decided** or **decided** process with the largest ID and this decision is consistent with all non-crashing processes
- **Liveness**: all processes participate and eventually either decide on a elected coordinator or crash

4.1 Bully Algorithm

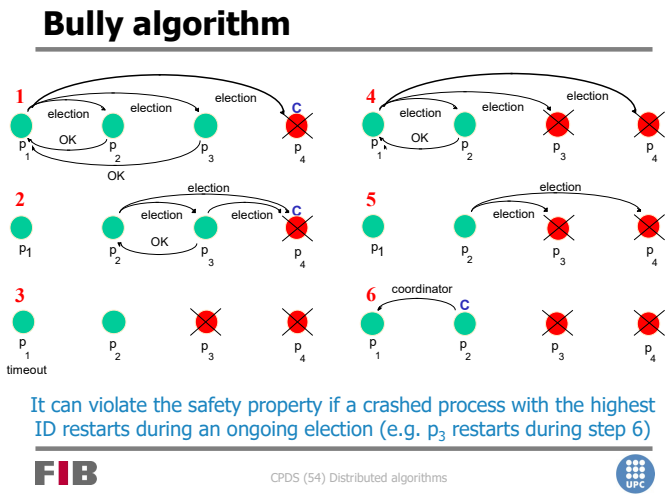
Assume:

- **Synchronous System**: can use timeout to detect process failure

- **Strongly Connected Topology**: exists a communication path between any two
- **Reliable Communication**: message delivery is reliable, no message drop
- **Full Knowledge of IDs**: each process know the ID of every other processes. (But doesn't know which one is down)
- **Simultaneity**: multiple processes may start elections concurrently
- **Crashed process cannot restart during an ongoing election**, otherwise the safety property will be violated

Algorithm

1. Some P_i starts election
2. P_i Send election message to all processes with **higher ID**
   - $\forall j > i$: Send$\big(P_j$, Election$\big)$
   - P_i: *"Hey, are you greater than me"?*
3. P_j receives election message, reply OK and starts another election (send messages to processes with higher ID) UNLESS it has already begun an election
   - NOTE: Replying OK and starting election message **simultaneously**
   - If P_j receives an election message, it must come from P_k : $k < j$.
   - P_k: *"Are you greater than me?"*
   - P_j: *"Yes, shut up"*
4. P_i receives OK, drop out of the election and wait for the Coordinator message
5. P_i does not receive any OK messages before timeout (synchronous system), it wins and sends the coordinator message to the rest of the processes
   - P_i: *"Are you greater than me? No one?"*
   - P_i: *"I'm the leader now"*



4.2 Chang and Robert's Ring Algorithm

Assume:

- **Asynchronous System**: not using latencies to detect process failures
- **Partial Knowledge of IDs**: Each process only knows its **successor** in the ring.
- **Process Cannot Fail During Election**: Otherwise liveness property is violated.
- Clockwise election

Algorithm

- Each process is marked as **non-participant**

- Every time a process sends/forwards a message, it becomes a **participant**
- P_i sends an election message E_i with its own ID (i) to its successor and become a **participant**
- P_j receives the Election message E_i (idea: forward $E_{\max(i,j)}$)
  - $i > j$: forward E_i to its successor (unconditionally forwarding larger election IDs)
  - $i < j$: (I'm larger, choose me!)
    - $\neg$ Participated(P_j) $\Rightarrow$: send $\boldsymbol{E_j}$ to its successor (notice it changed the candidate to itself)
    - Participated(P_j) $\Rightarrow$: don't forward
  - $\boldsymbol{i = j}$: (No one's larger than me) $\boldsymbol{P_j}$ **wins**, send the coordinator message C_i to its successor
- P_j receives the Coordinator message C_i ($i \neq j$), forward it to its successor and become a **non-participant**
- When the coordinator message reaches the coordinator, the leader discards the message, the election is over.

4.3 Enhanced Ring Algorithm

Each process must know **all nodes** in the ring
Each process add its ID to the election message.
When the message get back to the **Initiator**, choose the highest-ID process as the coordinator

Algorithm

1. P sends **Election message** to its **closest alive successor** (sequentially poll successors until one responds)
   - Question: are we still assuming asynchronous systems? because if we did, we cannot distinguish between a failed process and delayed message and we cannot rely on timeouts
2. At each step, **each process adds its ID to the list in the messages**
3. When the message get back to the **initiator** (the **FIRST** process that **detects its ID**), it elects as coordinator the process with the **highest ID** and sends a Coordinator message with that ID
   - Tricky: If P_2 first starts the election, it could be that when message reaches P_1 as $[P_2, P_3, P_4, P_1]$, P_2 crashes and the message is directly sent to P_3. Then P_3 became the first process to detect its own ID. Then it send the coordinator message.
4. **Each process add its ID to the message** when passing the Coordinator message
5. When the Coordinator message goes back to the initiator, if **the elected process is in the ID list**, we are done. Otherwise, **reinitiate the election**.

5 Multicast Communication

Types of multicase communication

1. **Multicast**: sned a message to a **process group**
2. **Reliable Multicast**: Deliver messages to **all** processes in a **(static) group or none** at all.
3. **Ordered Multicast**: Deliver messages while fulfilling ordering requirements
4. **Atomic Multicast**: Deliver messages to **all** processes in the **same order** in the presence of process failures **(dynamic process group)**

Desired Properties for a CORRECT process

- **Integrity**: Delivers every message **at MOST once** (filte duplicates)
- **Validity**: If m is multicasted, the process will **eventually** deliver m.
- **Agreement**: If a correct process delivers m, then **all other correct processes in the group** will eventually deliver m,

5.1 Reliable Multicast

This deal with the question: **Whether** a message can be delivered

5.1.1 Basic Reliable Multicast

Assume

- Processes **do not fail**
- Processes **do not join/leave the group** (static group)

Algorithm

Sender $p \in P$

1. p maintain a sequence number S_p
2. p stores a copy of the ongoing message into a **History Buffer**
3. p removes the copy when **everyone acknowledges the receipt**
   - Can keep a copy unboundedly long

Any **Receiver** $q \in P$

1. records $L_q(p_i)$ the number of last message from sender $p_i \in P$
2. $L_q(p_i)$ and S_p (S_p is from the message) comparison
   - $S_p = L_q(p_i) + 1$: new message
     - deliver the new message
     - reply **ack**
   - $S_p > L_q(p_i) + 1$: received missing messages
     - put message into the **Hold-back Queue**
     - reply **nack**, request a retransmission of missing messages
       - Retransmissions from the sender are also multicast messages
   - $S_p \leq L_q(p_i)$: already received, discard (because of multicast, we receive retransmitted messages)

5.1.2 Scalable Reliable Multicast

Use missing messages are reported using **NACK** s

1. **Multicast NACK** when missing messages
2. Each process wait a random delay prior to sending NACK (to sniff if others sent it)
   - supress NACK if other process sent it

Downside

- ↓: Sender should keep messages in history buffer **forever** (since no ACK present and we cannot know if everyone acked it)
  - in practice, deleted after a certain period
- ↓ Setting timers to ensure only one NACK is hard

5.2 Ordered Multicast

This deals with the question: **When** should the message be delivered
Messages can arrive in different order at different nodes due to network latency

5.2.1 FIFO Ordering (within each process)

Messages sent **from the same process** are **delivered in the order** by all processes
Our per-process hold-back queue guarantees this

5.2.2 Causal Ordering (between processes)

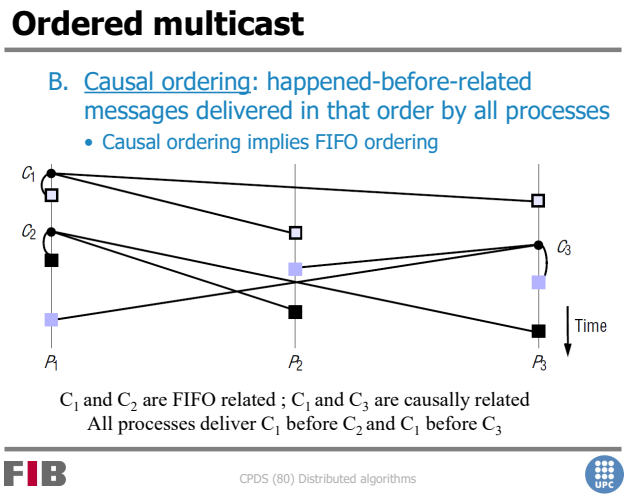
Happen-before-related messages are delivered in that order by all processes

- Causal order implies FIFO order (special case in the same process)

Implemented using vector clock (According to the solution of Q7, June 2021)

1. P_i (Sender) increases its own $\mathrm{VC}_{\mathrm{local}}[i] += 1$ ONLY WHEN sending the message
   - DO NOT update $\mathrm{VC}_{\mathrm{local}}[i]$ when deliver the same message
   - The message in the sender is delivered immediately (according to the solution of Q7)
2. A message is **delivered only if all causally preceeding messages have already been delivered**
   - $\left(\mathbf{VC}_{\mathbf{msg}}[i] = \mathbf{VC}_{\mathbf{local}}[i] + 1\right) \wedge \left(\forall k \neq i : \left(\mathbf{VC}_{\mathbf{msg}}[k] \leq \mathbf{VC}_{\mathbf{local}}[k]\right)\right)$
   - i: Sender PID
   - $\mathrm{VC}_{\mathrm{msg}}[k](k \neq i)$ is the sender's view when sending the message
   - $\mathrm{VC}_{\mathrm{local}}[k](k \neq i)$ is the receiver's view when the message is receives
   - Check after each delivery of the message (we may deliver multiple messages at a time from our hold-back queue)
3. Otherwise, postpone until the above-mentioned condition is met
4. P_j (Receivers) increase their own $\mathrm{VC}_{\mathrm{local}}[i]$ ONLY after **delivering** the message
   - DO NOT update when received the message
   - UPDATE the ***i*-th (sender)** index of the receiver, **NOT** the j-th.
   - DO **NOT take max** of the entire vector (as in Lamport Vector Clock), rather, only increment the **sender's index** in the local vector
     - Here, we track what which messages are **ACTUALLY DELIVERED** instead of some message you hear about from the sender but haven't actually seen and deliver it.
   - It means that the receiver have **DELIVERED** the messages until $\mathrm{VC}_{\mathrm{local}}[i]$ from the sender.

NO SOLUTION FOR 7, JUNE 2021



5.2.3 Total Ordering (all processes)

Using a **sequence number per group** (instead of per sender)

5.2.3.1 Sequencer Implementation

Sequencer: A single process assigning numbers

- Sender send the message to the sequencer

- The sequencer multicast the message with the number
- Receivers compare the number to choose whether to deliver the message or put it into the hold-back queue, and request for missing messages if necessary (scalable or basic?)

5.2.3.2 Joint Agreement Implementation

- Each process i keeps
  - A_i, P_i are stored as $(\text{scalar}, \text{PID})$
  - $A_i = (\text{scalar}, \text{winner PID})$ the largest agreed sequence number it received so far
  - $P_i = (\text{scalar}, \text{own PID})$ its own largest proposed number
  - Ordering: $(\text{scalar}_1, \text{PID}_1) > (\text{scalar}_2, \text{PID}_2)$
    $\Leftrightarrow (\text{scalar}_1 > \text{scalar}_2) \vee ((\text{scalar}_1 = \text{scalar}_2) \wedge (\text{PID}_1 > \text{PID}_2))$

Algorithm (According to the solution of Q7, Jan 2022)
Idea
- Sender → Receivers: Request their proposed numbers
- Receiver → Sender: Their proposed numbers
- Sender → Receivers: The chosen proposal

Steps
1. The **sender** i multicasts message m to **request** for the next
   1. Update its own proposed value $P_i \leftarrow (\max(P_i.\text{scalar}, A_i.\text{scalar}), i)$
   2. Add to the hold-back queue with the updated P_i: $\{m, \{\text{Proposed}, P_i\}\}$ where P_i is a tuple
   3. DO NOT include anything to the message (just sequence number request and the message, no P_i, the receivers don't need to know?)
2. When receiving the **request** message, each **receiver** j replies their own **proposed** seq number queue the message
   1. set the message's (local) sequence number as $P_j \leftarrow \big(\max\big(A_j.\text{scalar}, P_j.\text{scalar}\big) + 1, j\big)$
   2. Push it into the push-back queue as $\{m, \big\{\text{Proposed}, P_j\big\}$
3. The **sender** selects the largest of (**all proposals** N AND **its own** P_i) as the agreed number for m (blocking until all proposals arrive) and multicast the **agreement**
   1. $N = \big(\max\big(\big\{\forall j \in \text{Receivers} : P_j\big\} \cup \{P_i\}\big), \text{argmax}\big(\big\{\forall j \in \text{Receivers} : P_j\big\} \cup \{P_i\}\big)\big)$
   2. Update the sequence number of m to N in its own hold-back queue and sort the queue
   3. deliver all deliverable messages
4. When receiving the **agreement message**, each **receiver** j
   1. updates $A_j \leftarrow \big(\max\big(A_j.\text{scalar}, N.\text{scalar}\big), A_j.\text{scalar} \geq N.\text{scalar}? \, j : N.\text{PID}\big)$
   2. Update the sequence number of m to N in its own hold-back queue and **sort the queue**
   3. deliver all deliverable messages
5. **Deliverable messages**: Messages are delivered when it is at the **front of the queue and the number is agreed**.

NO SOLUTION FOR 5, JAN 2025

5.3 Atomic Multicast (View-synchronous Multicast / Virtual Synchrony)

With **FAULTY** processes
- Guarantee that a message is delivered to either **ALL** processes or none at all
- Delivered ONLY to the non-faulty members of the group
  - Processes have to agree on the current membership

Each message is associated with a view group: a message **belongs to** a view.

Different processes can change their view at different time (postpone the view change if necessary)

Important

- If a correct process delivers message in view V, the all other processes in view $V+1$ must also deliver m.
  1. Sender crashes before sending m, no one received it $\Rightarrow$ alright
  2. Sender sends m and crashes, one node delivered it
     - Other nodes must exchange m among themselves and **deliver** m **before** they are allowed to install the **new** view
- View change messages must be delivered in **total order** (sequencer or joint agreement)

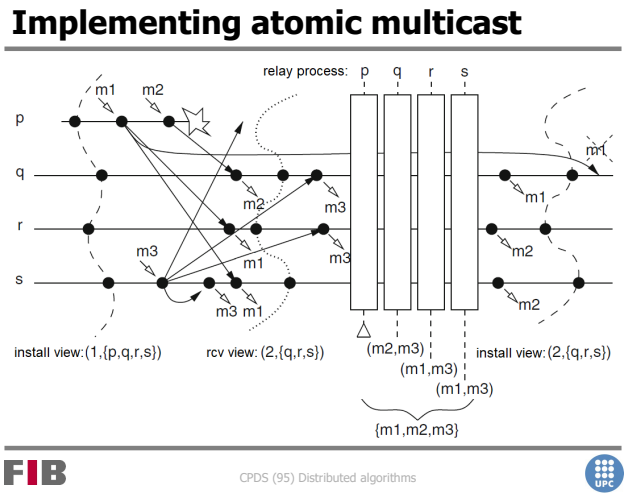
Normal Operations (within a view)

- Append the view number to multicast messages
- Check the message's view number when receiving
  - $m.\text{View} > \text{View}_{\text{local}}$: queue the message until the matching view is installed
  - $m.\text{View} = \text{View}_{\text{local}}$: deliver the message according to the specified multicast ordering
  - $m.\text{View} < \text{View}_{\text{local}}$: discard message

View Change

A **Flush** operation is triggered when a process P receives a **view change** message

1. Temporarily stop sending (from application) multicast messages
2. Forward to $\forall \boldsymbol{P_i} \in \textbf{View}$ all the messages P has **delivered**
   - As all processes in the view are doing this, we can make sure the we have the full set of the messages
3. If a process P discovered a new (missing) message from process $Q \in \text{View}_{\text{current}} \cap \text{View}_{\text{new}}$ (belongs to the current view), the process delivers this message
4. After received **ALL** the forwarded messages from **ALL** the processes ($Q \in \text{View}_{\text{current}} \cap \text{View}_{\text{new}}$) in the current view
   - Change the view (i.e. install new view)
   - Allow the application to resume multicasting messages



5.4 Consensus

We want to tolerate f failures

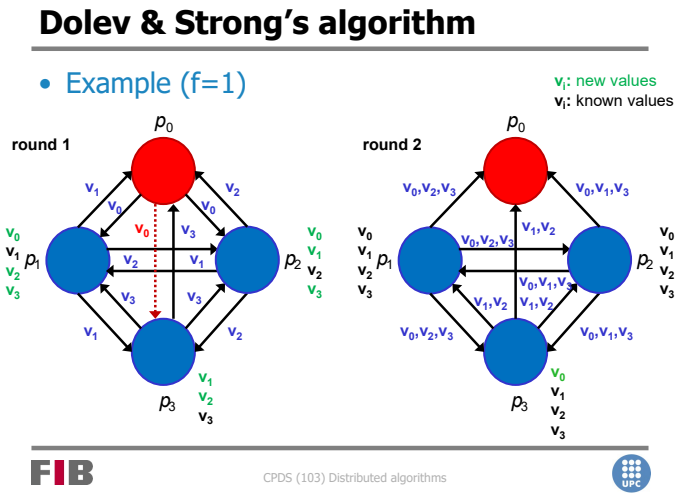
5.4.1 Unreliable Communication (Two-army Problem)

With unreliable communication, agreemet between two processes cannot be guaranteed

5.4.2 Crash-faulty Processes, Synchronous System

Assume: **Crash-faulty** processes, **reliable** communication, **synchronous** system.
Dolev & Strong's Algorithm
Algorithm has (f+1) rounds

1. Each process propose a value
2. From round 1 to f+1, each process
   1. multicast **new** values
      - new value: not sent in the previous round
      - in the first round, send the proposed value
   2. collect from all other processes and mark **new** values (to send in the next round)
   3. A round terminates when values for all processes are collected or by timeout (based on mi message latency)
3. Each process decide against collected values



5.4.3 Byzantine-faulty (Lying) Processes, Synchronous System

Assume Byzantine-faulty processes, reliable communication, **synchronous** system
We restrict the problem to **ONE** general sending his value to others:
A **commanding general** send must send a message to his n-1 **lieutenants**
Interactive Consistency Requirements

- **IC1 (Agreement)**: All **loyal** lieutenant obey the same order (reach agreement)
- **IC2 (Validity)**: **If the commander is loyal**, then all loyal lieutenant obey the order he sends
  - If the commander is **NOT loyal**, we only need to reach consensus, but the value doesn't matter (special value $\perp$ is also considered valid if no majority)
  - From previous exams: When the commander is a traitor, the loyal lieutenants can agree an undefined value $\perp$.

Impossible Result: with 3 processes, cannot traitor even 1 traitor:
Because we cannot distinguish if the commander is the traitor or the other lieutenant is the traitor.

- Imagine Lieutenant **L1** is loyal and
  - **C -> L1**: attack

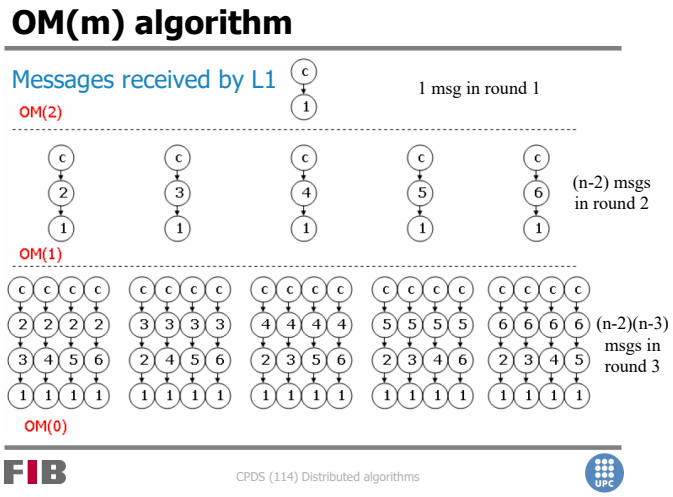
  - **L1 -> L2**: attack
  - **L2 -> L1**: retrieve
- Possibilities
  1. **C** is the traitor: **C -> L1 (Attack)**, **C -> L2 (Retrieve)**
  2. **L2** is the traitor: **C -> L2 (Attack)**, **L1 -> L2 (Attack)**, **L2 -> L1 (Retrieve)**

Bounds
- **Without Crypto**: Need $N > 3m + 1$ processes to tolerate m traitors

5.4.3.1 Oral Message Algorithm (OM) !!!!

Cross-examining witnesses by exausively ask everyone what the commander told then and what everyone told everyone.
Tolerate at most m traitors with at least $\mathbf{3m + 1}$ **participants**
Assumption
- Sent messages are correctly delivered (no corruption)
- The absence of message can be detected
- The receiver of a message knows who sends it

In the following figure, lieutenant 1 wants to know what the commander told him. He build a graph of every possible message flow from the commander (C) to himself (1).
And this process is repeated for every lieutenant during the recursion (this is just a local view of lieutenant 1).



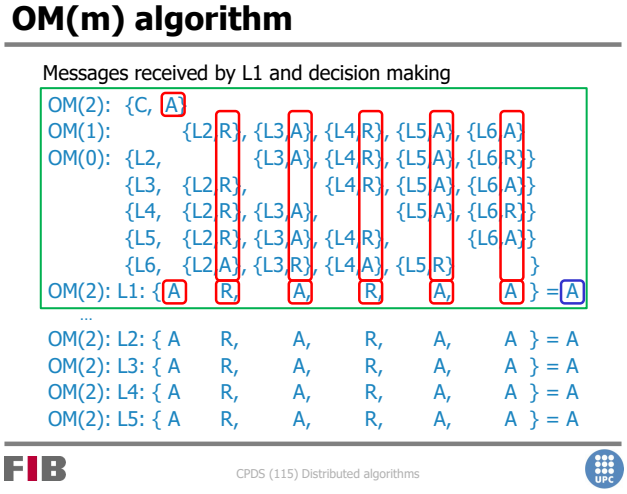
Assume:
1. **Fault Tolerance**: At most m unreachable or illoyal participants
2. **Network**:
   1. No message loss
   2. The receiver knows the sender of the message
3. **Timing**: Absence of a message can be detected

Algorithm (Recursive):
OM$(n, t), t > 0$:
1. The commander send its value to every lieutenant
2. For each lieutenant i, let v_i be the value received from his commander
   - i then act as a commander in OM$(n-1, t-1)$ in which he send his own v_i to every **other** participants.

3. $\forall i, j (i \neq j)$: let v_j be the value in which lieutenant i received from j in step $\text{OM}(n-1, t-1)$
   - i choose majority$(v_1, ..., v_{n-1})$.

The following figure shows how consensus is reached (2 traitors)

- In the green box, it shows how L1 determined determines what its commander C say
  - Layers of indirection
    - $\text{OM}(2)$: Direct message from his commander C, but maybe C is a traitor
    - $\text{OM}(1)$: One layer of indirection: What does C say to each of other Lieutenants. But what if some lieutenant are traitors?
    - $\text{OM}(0)$: Two layers of indirection: What does each lieutenant says about each other lieutenant says about the commander
  - Values received
    - First column: The commander told me to attack
    - Second Column:
      - L2 told me the commander told me to retrieve, and
      - All other lieutenants said L2 told them to retrieve
    - Third Column:
      - L3 told me to attack
      - L2-6 said L3 told them to retrieve and L6 said otherwise
    - ...
  - L1 needs to decide what each lieutenant said about the commander
- The cluster needs to decide what the commander



5.4.4 Faulty Process, Asynchronous System

NO algorithm can **guarantee** to reach a consensus (even with one faulty process)
Consensus can be reached with **some possibility**

5.4.4.1 Fault Masking + Basic Algorithm

- Assume failed process **always** recover and can **reintegrate** into the group.
- A round terminates when **every** expected messages is **received** (if a process is not responding, wait longer)

5.4.4.2 Failure Detectors + Synchronous Algorithm

- Determines which process has crashed
- A round terminates when
  - **every** expected message is received, or
  - **failure detector** suspect its sender's down
- Consensus with failure detectors requires that
  - Every faulty process is **permanently suspected** at least by some correct processes
  - At least a correct process is never suspected
  - Note: doesn't always hold for asynchronous systems but long enough for correct process to decide

5.4.4.3 PAXOS (Quorum)

Assume:

- Processes operate at **arbitrary speed**, **may crash**
- Restarted processes remember where in the process they were
- Messages take arbitrary long time to deliver, get lost or duplicated, but **NOT corrupted**

A single process can take multiple roles
Proposers: Propose values to agree on a single one among all proposed values
Acceptors: Decide whether to accept proposed values (act as the memory of the system)
Learners: Check if any value has been chosen

Phase 1

- Proposer: send **prepare** requests to **each acceptors** with seq. nr. N
  - "Please do not vote for any proposal number less than N"
- Acceptor: $N > P_{\max}$:
  - Promise not to vote for any proposals with seq. nr. $< N$
    - $P_{\max} \leftarrow \max(P_{\max}, N)$
  - Reply with the highest-numbered **voted** proposal (seq, val)

Phase 2

- Proposer: collect **promises** from the **majority** of acceptors
  1. Choose the value with the highest-numbered proposal among the responses
  2. If no proposal in the response, use its own value
  3. send **accept** messages to acceptors
- Acceptor:
  1. **Vote** for proposal **unless** it has **already promised** a number greater than N.
  2. Acknowledge the proposer and leaners

Phase 3

1. When leaners collect notifications from a **majority of acceptors** for a given proposal, it becomes the final decision.
2. Each leaners may take action and inform the client about the outcome

Best Case Message Count $6f + 4 = \underbrace{(2f+1)}_{\text{Propose}} + \underbrace{(f+1)}_{\text{Reply}} + \underbrace{(2f+1)}_{\text{Accept}} + \underbrace{(f+1)}_{\text{Reply}}$

6 Concurrency

Conflicting Operations: (when two transaction's lifespan overlaps)
Two transactions are conflicting if their effect depends on the order in which they execute

- **Read-Write Conflict**: A transactions is reading a value another transaction is updating

- **Write-Write Conflict**: Two transaction both attempt to update the same value

Strict Execution of Transactions (Avoid dirty read and premature writes)
Both **read and write** operations on an object must be **delayed** until **earlier transactions** that **wrote** on that object has been **committed or aborted**

6.1 Serially Equivalent

- two transactions are serially equivalent when **all pairs** of **conflicting operation** are executed in the **same order** at **all shared objects**.
- i.e. **the same transaction always operates first** (when executing **conflicting operations**)

6.2 Strict Two-phase Locking

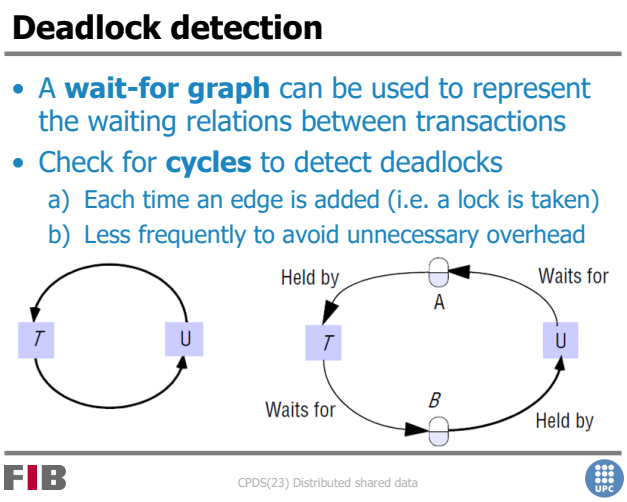
Transaction Ordering: No predefined order of the transaction
- **Serial Equivalence**: A transaction is not allowed new locks after it has released a lock
- **Strict Execution**: Lock must be held until the transaction commits or aborts

Increase Concurrency: Use read and write lock
- **Read lock**: if **no writer**, multiple transactions can take read lock
- **Write lock**: if **no reader nor writer**, one transaction can take the write lock
- Allow read lock to be **promoted** to write lock when there is **only one reader**, namely the read lock holder

Algorithm
- On read/write operation
  - Object not locked: lock the object and proceed
  - Object locked
    - Conflicting Lock: wait until it's unlocked
    - Non-conflicting lock: share the lock (e.g. multiple read lock)
  - Object lock promotion (e.g. read-to-write lock promotion)
    - Can promote: promote and proceed
    - Cannot promote, prevented by conflicting lock, wait until unlocked
- On commit/abort: Unlock all object for this transaction

6.2.1 Deadlock Detection

Wait-for Graph
- $A \rightarrow B$: A waits for B
- Check for deadlock when
  - Each time an edge is added (i.e. a lock is taken)
  - May be less frequently to avoid overhead
- Deadlock Resolution
  - Abort transaction involved in the cycle
    - Which to abort: age of transaction, number of cycles (in deadlock) the transaction is involved in, etc.
  - Lock timeouts
    - A taken lock is made vulnerable after a timeout
    - Lock is broke if any other transacting is waiting on it
    - Transaction with broken lock is aborted
    - $\downarrow$ unnecessary aborts. How to choose timeout?



6.3 Timestamp Ordering

Each transaction has a unique timestamp
- assigned **when the transaction starts**
- defines its position in the sequence of transactions

Each Object x keeps
- Write timestamp for its **committed** value: $\text{wts}(x)$
  - Uncommitted transactions keeps a tentative value and $\text{wts}(x)$ is kept unchanged
- Read timestamps (highest): $\text{rts}(x)$
- Pending read and tentative write ops with their corresponding timestamps

Validation
- **Each operation is validated** when they are **performed**
- If a transaction made it to the commit stage, we should always commit because each of its operations is legal

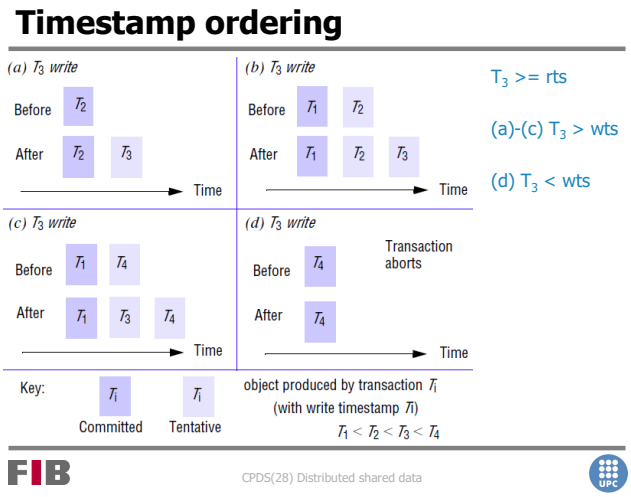
6.3.1 Write Validity

When transaction T wants to write to x, it is valid only if
x was last read or written by the earlier transactions (and can be written by T as well)

We then **add this write to tentative versions**, update the version if it already exists

$$\text{ts}(T) \geq \text{rts}(x) \wedge \text{ts}(T) > \text{wts}(x) \tag{3}$$

Otherwise we would have $\text{ts}(T) \leq \text{wts}(x) \vee \text{ts}(T) < \text{rts}(x)$
- $\text{ts}(T) < \text{wts}(x)$: We overwrite a "future" value written by later transactions (because we predefine the order when the transaction starts)
- $\mathbf{ts(T) < rts(w)}$: We "altered the history" because a "future" transaction read the value we are overwriting and thinks no one altered it.
- $\text{ts}(T) = \text{wts}(w)$: not possible because T is not committed



In the fugure above, (a)-(c) shows $\text{wts}(x) < \text{ts}(T_3)$, while (d) shows $\text{wts}(x) > \text{ts}(T_3)$, which is unacceptable.

6.3.2 Read Validity

When transaction T wants to read from x, it is valid only if x is **last written by earlier transactions** (No later/future transaction write to it).

$$\text{ts}(T) > \text{wts}(x) \tag{4}$$

Note that no version of $\text{ts}(T) = \text{wts}(x)$ because the write for uncommitted T is stored as **tentative version** of the value and **not committed**

Read most recent version: $(\max\{t : \text{wts}(x) \leq t < \text{ts}(T)\})$

1. It is a **tentative version**
   - Pend until this tentative version is committed, i.e. the (logically) earlier transaction completes
   - Scenario: It may happen that some tentative versions is newer than the committed version, we need to read the latest version
2. It is **committed**
   - Read and update timestamp to $\text{ts}(x) = \max\{\text{rts}(x), \text{ts}(T)\}$
3. T **already have a transitive version**, use it (Read your writes)

6.4 Optimistic Concurrency Control

Phases

- **Working Phase**: Operation proceeds, write to a private workspace
  - Keep tentative versions of each object it updates
  - Keep read and write set of the transaction
  - Read operations are directed to the tentative versions (if exists) or committed ones
  - Write operations are directed only to tentative versions
- **Validation Phase**: Check if it has conflicted with concurrent transactions
  - A transaction is **given a sequence number when entering validation phase**
  - NOTE: Different from timestamp ordering where the sequence number is given at the start of the transaction.
- **Update Phase**: Tentative versions of any write operations are committed to the database

6.5 Backward Validation

Check if the validating transaction (T_v) read **stale** data.

Compare the **read set** of T_v with **write set** of transactions **committed while T_v is working**

NOTE: **DO NOT** compare **ABORTED** transactions.

$$\mathrm{RS}(T_v) \cap \left(\bigcup_{\substack{T_{\text{start}} < T_i \leq T_{\text{end}} \\ \wedge\ \text{Committed}(T_i)}} \mathrm{WS}(T_i) \right) = \varnothing \tag{5}$$

6.6 Forward Validation

Check if the validating transaction (T_v) **alters active (UNCOMMITTED) transactions' view of the data**

$$\mathrm{WS}(T_v) \cap \left(\bigcup_{T_i \in \text{ Active}} \mathrm{RS}(T_i) \right) = \varnothing \tag{6}$$

↑ More flexible conflict resolution
- Abort the committing transaction
- Abort the conflicting active transactions
- Let conflicting transactions finish and try again (they may abort)
- ...

↓ Dynamic read sets when validating
- **Block read operations** of active transactions on **contended object** while in **validation and update phase**

7 Distributed Transactions

Transactions can access objects managed by multiple servers
- For each transaction, one server can act as the coordinator
  - Coordinator is responsible for opening and closing (commit/abort) of a transaction
- Each server accessed by a transaction is a participant (the coordinator can also be the participant in the same transaction)

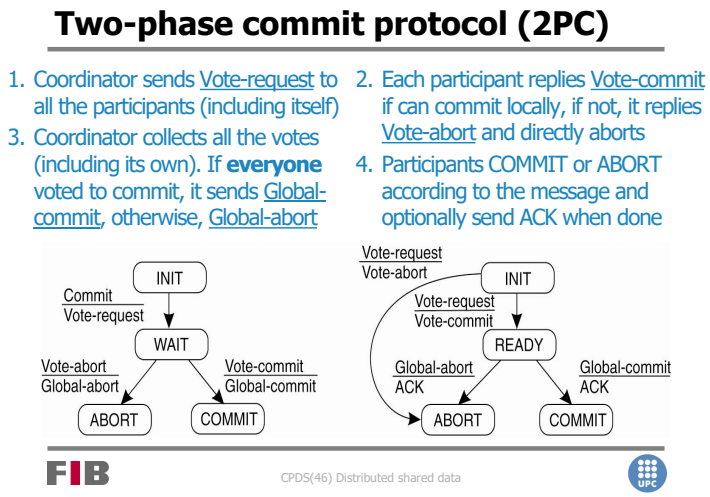
7.1 Distributed Commit

7.1.1 Two-Phase Commit Protocol

Abort whenever **any** participant decides to abort.

1. Coordinator send **vote-request** message to all participants (including itself)
   - Coordinator **blocked** on **WAIT** state to **collect votes** from every participants (including itself)
     - On timeout, ask the participant to send the vote again; if no response, abort
2. Participants reply
   - **vote-abort** if they cannot commit locally
     - Directly **abort**
   - **vote-commit** if they can commit locally
     - If it votes commit, it must **ensure** that it will **eventually** be able to carry out the commit [write-ahead log perhaps?]

    - **Blocked** on **READY** state to see if the coordinator decides to abort
      - On timeout
        - ask the coordinator to resend the global decision
        - query other participants about their state
          - if any of them decides to abort, we should abort
          - if no response or all participants blocked on **READY**, we cannot decide
3. Coordinator collect **all votes** (including itself), if
   - **ANY** says abort, send **global-abort**
   - otherwise, **ALL** says commit, send **global-commit**
4. Participants commit or abort according to the message
   - optionally send acknowledgement when done



In the above figure, the coordinator state machine is on the bottom left, while the participant state machine is on the bottom right.

7.1.2 Other Protocols

Three-Phase Commit: avoids blocking problem but adds latencies and does not tolerate network partitions. → Often not applied in practive.

Paxos Commit: 2PC on top of Paxos

7.2 Distributed Concurrency Control

Schedule concurrent transactions to preserve serial equivalence.

- Each server is responsible for the concurrency control of **its own objects**
- All Servers must agree on **the same order of transactions** to preserve serial equivalence (recall that serial equivalence means that if T is before U in their access to objects in one server, then **all server** should have T before U).

7.2.1 Strict Two-Phase Locking (Distributed)

For each object, locks are **maintained locally** on its server
Locks are held until the transaction is committed or aborted at **all servers involved**.

7.2.1.1 Distributed Deadlock Detection

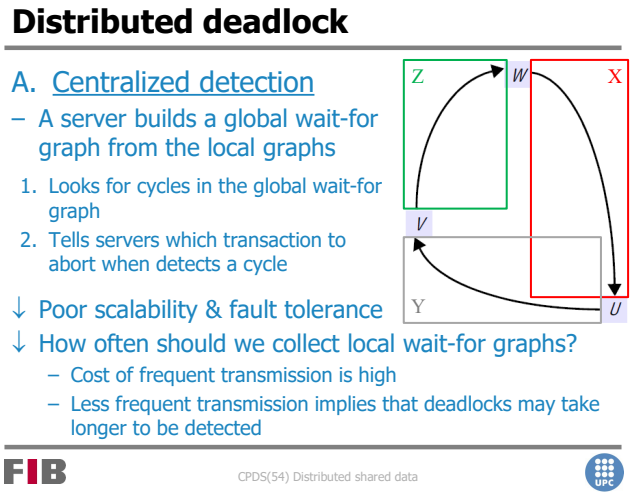
7.2.1.1.1 Centralised Detection

A server builds (merges) a **global wait-for graph** from local wait-for graphs

- Looks for cycles in the global wait-for graph
- Tells servers which transaction to abort

Cons

- ↓ Poor scalability and fault tolerance
- ↓ How often should we **collect** local wait-for graphs?
  - cost of frequent transmission is high
  - Less frequent transaction implies deadlock may take longer to detect

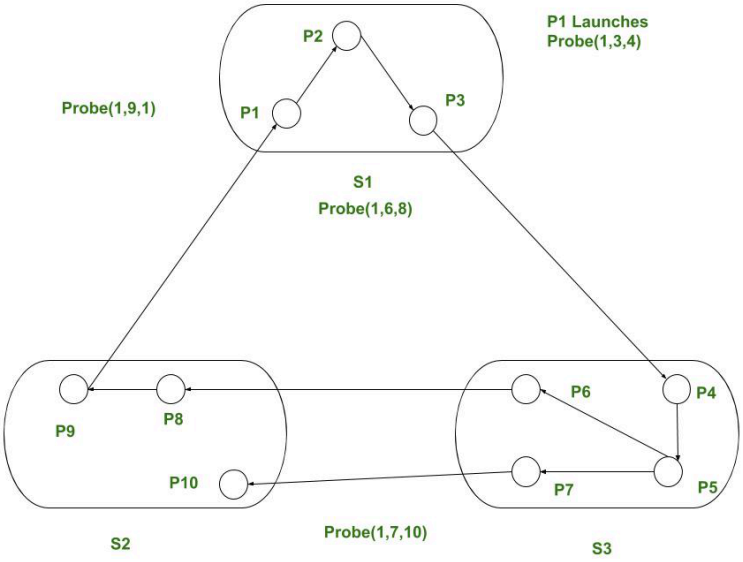


7.2.1.1.2 Edge Chasing

When we want to detect a specific dead lock, we send a **probe** message to the next "site" following the dependencies.

$$\text{Probe}\left(\underbrace{P_{\text{initiator}}, P_{\text{sender}}}_{\text{current site}}, \underbrace{P_{\text{receiver}}}_{\text{next site}}\right) \tag{7}$$

If the probe message arrive to the same site and $P_{\text{initiator}} = P_{\text{receiver}}$, we have a deadlock. The following figure shows a deadlock detection process. $\text{Probe}(1, 6, 8)$ is sent from $P_6 \in S_3$ to $P_8 \in S_2$. And when the probe message get back to site S_1, we have $P_{\text{initiator}} = P_{\text{receiver}} = 1$.



7.2.2 Timestamp Ordering (Distributed)

The coordinator of a transaction assigns it a **globally unique timestamp** when it starts. As different servers can act as coordinators, they must agree on the order of the timestamp they generate (e.g. as $(\text{ts}_{\text{local}}, \text{ServerID})$).

Locally to each server, the protocol acts the same

7.2.3 Optimistic Concurrency Control (Distributed)

A transaction is validated by a **collection of servers** of the object it accessed.

Problem: Different servers may serialise the same set of transactions in different orders.

Backward Validation (Graph Validation)
1. Perform **local validation**
2. **check globally** if **the combination of the ordering** is **serialisable**
   - Transaction being validated is not involved in a cycle
   - e.g. $T_1 \to T_2$ on server A but $T_2 \to T_1$ on server B

Forward Validation (Global Sequencing)
- Coordinator assigns to the transaction a **global sequence number** at the start of validation
- All servers **must** serialise in this specific order, otherwise abort.

8 Consistency

8.1 Consistency Model: Distributed Data Store

1. Distributed across multiple nodes in a **raplicated** fassion
2. Each replica holds **a copy of the entire store**
3. Each process can access **one or several replicas**
4. Write operations to a replica are **propogated** to other replicas

Notation:
1. $W_i(x)a$: Write by process $P_i \in P$ to item x with value a, $(x \leftarrow a)$

2. $R_i(x)b$: Read by process $P_i \in P$ from item x with a return value of b, $(b \leftarrow x)$

8.2 Data-centric Consistency

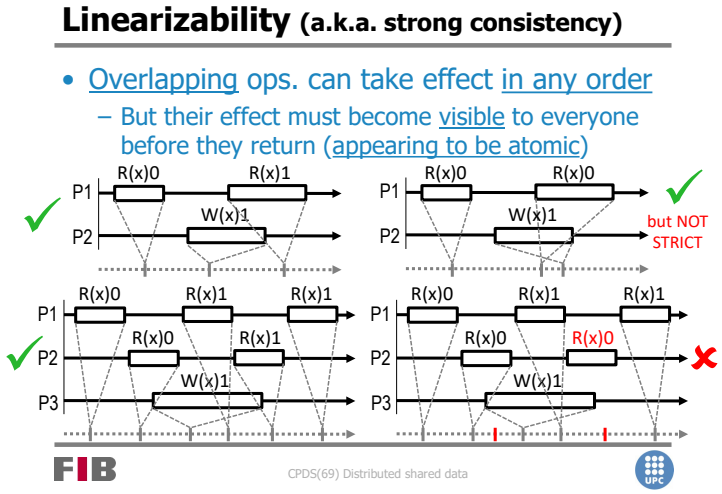
8.2.1 Strict Consistency

1. Exclusive Ordered Writes
   - Given 2 concurrent writes on x, one must go first
2. Up-to-date Monotonic Reads
   - A read on x returns the **most recent** write on x **regardless of the use of replica** by the client
   - Subsequent reads on x returns the same value or some later value, **regardless of the use of replica** by the client
3. Instant Visibility
   - A write by any processor needs to be seen instantaneously by all other processors
   - Impossible in distributed systems
     - Network latency prevents instant update
     - No perfectly synchronised physical clock

8.2.2 Linearisability (Strong Consistency)

1. Same goal as Strict Consistency
   1. Exclusive Ordered Writes
   2. Up-to-date Reads
   3. Overlap: Operations can overlap
2. Linearsation: Operation should appear to **take effect instantly** (update visible to all processes) **at some point** during its invocation (start) and reponse (end).
3. Compositional: **Separated linearisable histories** of objects can be **composed** in a linearisable history.

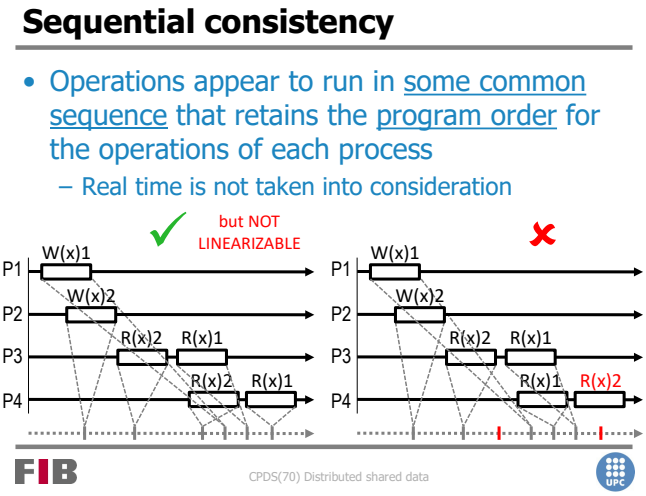
It follows that for non-overlapping operations, if A finished before B starts (in read time), B should see the changes made in A.



In the figure, $R(x)0$ is contradictory since it is after $R(x)1$ after which $W(x)1$ has already taken place.

8.2.3 Sequential Consistency

1. Processes agree on a global order that retains the programme order for each process
   - If in P_1, the order is $< W(x)_1, R(x)_1 >$, the global order cannot be $< R(x)_1, ..., W(x)_1 >$
   - Does not guarantee real-time ordering. If P_1 writes $x \leftarrow 1$ and finishes, then P_2 reads x two seconds later, it may see the old value. But it is accepted as long as **all processes** agree on the same history
2. Not compositional.



We can see an example. We try to make a global order of the execution of the above schedule:

1. $W(x)_2 2 < W(x)_1 1$
   1. P_3 programme order: $R(x)_3 2 < R(x)_3 1$
   2. Read order: $W(x)_2 2 < R(x)_3 2 \wedge W(x)_1 1 < R(x)_3 1$
   3. $\Rightarrow W(x)_1 2 < R(x)_3 2 < R(x)_3 1$
   4. It is only possible when $W(x)_2 2 < W(x)_1 1$. Otherwise $W(x)1 < W(x)2 < R(x)2 < R(x)1$ is FALSE.
2. $W(x)_1 1 < W(x)_2 2$
   1. P_4 programme order: $R(x)_4 1 < R(x)_4 1$
   2. read order
3. Contradictory

8.3 Client-centric Consistency

- Guarantees consistency for **SINGLE CLIENT**'s access to the data store. (client's own view)
- Does not guarantee consistency by multiple different clients (Unlike data-centric)
- Implemented by using version vectors to keep track read and write sets for each client
  - write set (WS_i): **Writes** performed by the client itself
  - read set (RS_i): **Writes** that are **visible** during client's **previous reads**

The Algorithm

On Replica(Server) R_i

Algorithm 1: ReplicaRead(RS_i, WS_i, C_i)

```
1 if (strategy = "ReadMyWrite") then
2   | while VV_k < WS_i do wait; end
3 else if (strategy = "MonotinicReads") then
4   | while VV_k < RS_i do wait; end
5 end
6 val ← doRead();
7 C_i.reply(val, VV_k);
```

Algorithm 2: ReplicaWrite(RS_i, WS_i, C_i)

```
1 if (strategy = "MonotinicWrites") then
2   | while VV_k < WS_i do wait; end
3 else if (strategy = "WritesFollowReads") then
4   | while VV_k < RS_i wait; end
5 end
6 val ← doRead();
7 C_i.reply(val, VV_k);
```

On Client

Note that client read and write blocks until it receives the reply from the replica R_k with (the replica's) version vector VV_k.

Algorithm 3: ClientRead(RS_i, WS_i, R_k: Replica)

```
1 R_k.requestRead(RS_i, WS_i);
2 VV_k ← R_k.recv();
3 RS_i ← max(VV_k, RS_i);
```

Algorithm 4: ClientWrite(RS_i, WS_i, R_k: Replica)

```
1 R_k.requestWrite(RS_i, WS_i);
2 VV_k ← R_k.recv();
3 WS_i ← max(VV_k, WS_i);
```

8.3.1 Read Your Writes (RYW)

If the client write data item x on replica R_k, any subsequent reads **by client C on replica R_k** will see (return at least as recent as) the write.

Read(RS_i, WS_i): Wait until $\mathrm{VV}_k \geq \mathrm{WS}_i$.

Wait until the replica R_k has seen client's write WS_i.

8.3.2 Monotonic Reads (MR)

If the client C reads data item "x" on replica R_k, any subsequent reads **by client C on replica R_k** will return at least as recent as the read.

Read($\mathbf{RS_i}$, $\mathbf{WS_i}$): Wait until $\mathrm{VV}_k \geq \mathrm{RS}_i$.

Wait until the replica R_k has more (or same) information as the client's reads RS_i

8.3.3 Monotonic Writes (MR)

Write($\mathbf{RS_i}$, $\mathbf{WS_i}$): Wait until $\mathrm{VV}_k \geq \mathrm{WS}_i$. Wait until the replica R_k has registered the client's writes WS_i.

8.3.4 Writes Follow Reads (WFR)

Write($\mathbf{RS_i}$, $\mathbf{WS_i}$): Wait until $\mathrm{VV}_k \geq \mathrm{RS}_i$ Wait until the replica R_k has seen all the reads from the client's write

8.4 Consistency Protocol

8.4.1 Primary-based Protocols

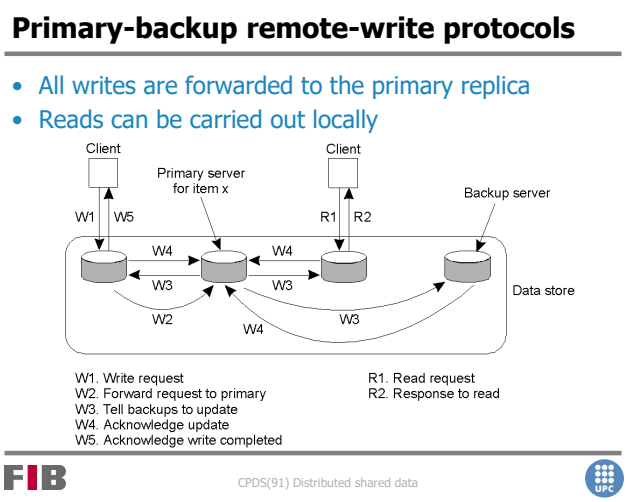
Writes are managed by a **single primary replica**

8.4.1.1 Primary-backup Remote-write

1. All **Writes** are forwarded to **the** primary replica
   - Blocking write protocol is used.
2. Reads can be carried out locally
3. On failure, leader election

Features

- ↓ Bad write performance: blocking write protocol
  - non-blocking alternative: if some replicas fail during update, read-your-writes consistency is not guaranteed
- ↑ Easy to implement sequential consistency
  - Primary sends write operations to replicas using **FIFO-ordered view-synchronous atomic multicast**
- Implements linearisability if read requests are also forwarded to the primary

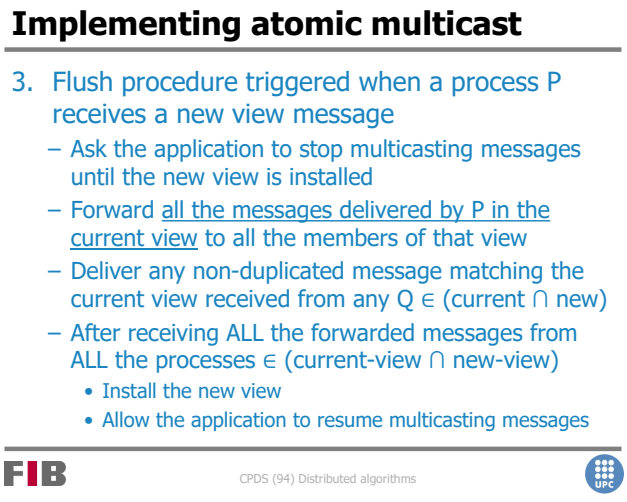


8.4.1.2 Primary-backup Local-write

E.g. Used in distributed file systems

1. Primary migrates to the replica that is writing
2. Successive writing are carries out locally
3. Replicas are updated using a non-blocking protocol
   - Optimises for several writes (only propagate after the last write of the client)
   - All replicas receives update in the same order? -> Sequential consistency

Q: What kind of consistency does this provide?



8.4.2 Replicated-write Protocols

8.4.2.1 Active Replication Protocol

Clients broadcast each operation to **all** replicas and operations are carried out in the same order

- in **Total-FIFO-ordered multicast**
- **Sender waits** until the multicast message is delivered back to the client

Consistency

- Multicasts read and writes: linearisability

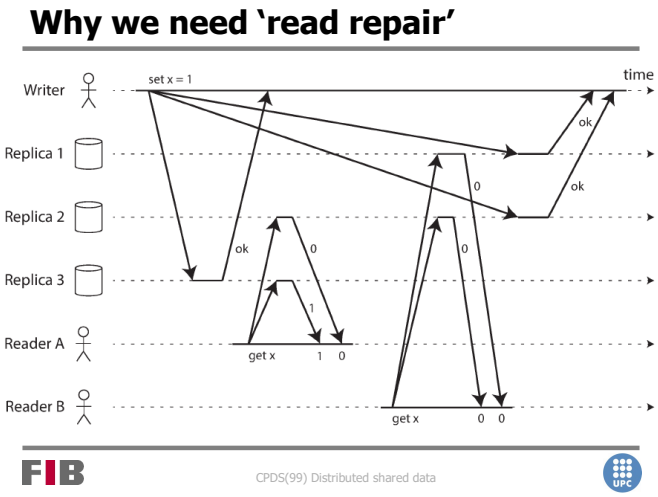
- Multicasts only writes: **sequential consistency**
  - Read returns the current value of the local copy

8.4.2.2 Quorum-Based Protocols

Clients must get replies from a **quorum** (N_R, N_W) of N replicas before reading or writing
- Avoid write-write conflict: $N_R > \frac{N}{2}$
- Avoid read-write conflict: $N_R + N_W > N$

To implement linearisability:
- use **majority quorums** for reading and writing: $N_R > \frac{N}{2}$, $N_W > \frac{N}{2}$
- use **version vectors** to achieve ordered writes according to their real-time order
  - Writes must read first to learn the highest version and potential siblings
- Perform **read-repair** to achieve up-to-date monotonic reads
  - read operations must write to update stale replicas

Read Repair



8.4.2.2.1 Quorum Write

1. Read Phase
   - Send read requests to **ALL** (or N_R) replicas
   - Await replies from N_R replicas
   - Learn the highest version among the replicas, merge siblings if needed
2. Write Phase
   - Send write requests (containing the version learnt) to all (or N_W) replicas
     - If we send to N_W replicas, the remaining replicas are updated in the background
   - Await acknowledges from N_W replicas
   - Write is complete and return to the user

8.4.2.2.2 Quorum Read

1. Read Phase
   - Send read requests to **ALL** (or N_R) replicas
   - Await replies from N_R replicas
   - If version numbers are the same, return the value to the user
2. Write Phase (Read Repair)
   - Send write requests to **ALL** (or outdated) replicas

    - The requests contains the largest version number and associated up-to-data value
- Await acknowledgement from N_W (or only updated) replicas
- Return the value to the user